ЛР4. Разведочный анализ данных (EDA) — пошаговый гайд

Результат — ноутбук с разделами 2.1–2.7 методички. Методичка рекомендует взять набор, полученный в ЛР3: ниже используется `melb_clean.csv` (Melbourne Housing после обработки пропусков и выбросов). Цель — `Price`.

Что сдавать

Раздел	Содержание	Минимум
2.1	Описание, типы признаков, описательные и частотные характеристики	все признаки
2.2	Кодирование **каждого** категориального признака + обоснование, таблица «признак — тип — способ»	таблица
2.3	≥ 2 новых признака из смысла данных + интерпретация	2 признака
2.4	Диаграммы рассеяния цели от каждого непрерывного признака; ≥ 2 линеаризующих преобразования, до/после	2 признака
2.5–2.6	Матрицы Пирсона и Спирмена, визуализация, где различаются, связь с целью, проверка гипотез 2.3–2.4	2 матрицы
2.7	≥ 2 признака разного масштаба × ≥ 2 метода; плотности, статистики; выбор метода и применение	графики + вывод

Как создать ноутбук и оформлять ячейки — см. раздел 0 гайда ЛР3. Понадобятся `pip install scikit-learn seaborn`.

2.1. Первичный анализ признакового пространства

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import (OneHotEncoder, TargetEncoder, StandardScaler,
                                   MinMaxScaler, RobustScaler, MaxAbsScaler)

df = pd.read_csv('melb_clean.csv', parse_dates=['Date'])
TARGET = 'Price'
print(df.shape)
df.dtypes
```

```
num_cols = ['Rooms', 'Distance', 'Bedroom2', 'Bathroom', 'Car', 'Landsize',
            'BuildingArea', 'YearBuilt', 'Lattitude', 'Longtitude', 'Propertycount']
cat_cols = ['Type', 'Method', 'Regionname', 'CouncilArea', 'Suburb', 'SellerG']
display(df[num_cols + [TARGET]].describe().T.round(2))
```

```
pd.DataFrame({'уникальных': df[cat_cols].nunique(),
              'топ-категория': df[cat_cols].mode().iloc[0],
              'доля топа': [df[c].value_counts(normalize=True).iloc[0] for c in
cat_cols]}).round(3)
```

В Markdown-ячейке — таблица типов (номинальный, порядковый, количественный) и короткое описание предметной области (как в ЛР3).

2.2. Кодирование категориальных признаков

Правило выбора способа:

Признак	Тип	Категорий	Способ	Почему
`Type`	номинальный	3	One-Hot (drop first)	мало категорий, порядка нет
`Method`	номинальный	5	One-Hot (drop first)	мало категорий
`Regionname`	номинальный	8	One-Hot (drop first)	мало категорий
`CouncilArea`	номинальный	≈ 33	Frequency	one-hot дал бы 30+ разреженных столбцов
`Suburb`	номинальный	≈ 300	Target encoding (с кросс-фиттингом)	много категорий, сильная связь с ценой
`SellerG`	номинальный	≈ 250	Frequency	размер агентства как признак

`drop_first` убирает один столбец, чтобы не было **ловушки фиктивных переменных**: сумма всех one-hot столбцов равна 1, то есть равна столбцу свободного члена — признаки линейно зависимы (см. теорию).

```
enc = df.copy()

# One-Hot
ohe_cols = ['Type', 'Method', 'Regionname']
enc = pd.get_dummies(enc, columns=ohe_cols, drop_first=True, dtype=int)

# Frequency encoding: доля категории в выборке
for c in ['CouncilArea', 'SellerG']:
    enc[c + '_freq'] = enc[c].map(enc[c].value_counts(normalize=True))

# Target encoding: среднее ln(Price) по пригороду, сглаженное и посчитанное вне фолда
te = TargetEncoder(target_type='continuous', smooth='auto', cv=5, random_state=0)
enc['Suburb_te'] = te.fit_transform(enc[['Suburb']], np.log(enc[TARGET])).ravel()

enc = enc.drop(columns=['CouncilArea', 'SellerG', 'Suburb', 'Address'], errors='ignore')
enc.filter(regex='Type_|Method_|Region|_freq|_te').head()
```

> **Target encoding и утечка**
>
> `TargetEncoder.fit_transform` кодирует каждую строку по фолдам, в которые она **не** входит (кросс-фиттинг). Наивное `df.groupby('Suburb')[TARGET].transform('mean')` включает в среднее собственную цену объекта — это утечка. В ЛР5–ЛР6 кодировщик обучают только на train, а к test применяют `te.transform`.

2.3. Предметно обусловленные признаки

```
enc['Age'] = enc['Date'].dt.year - enc['YearBuilt']                    # возраст дома на момент продажи
enc['AreaPerRoom'] = enc['BuildingArea'] / enc['Rooms']                # м² на комнату
enc['BathPerRoom'] = enc['Bathroom'] / enc['Rooms']                    # «комфортность»
enc['BuildToLand'] = enc['BuildingArea'] / enc['Landsize'].replace(0, np.nan)  # плотность застройки
enc['SaleMonth'] = enc['Date'].dt.month
enc[['Age', 'AreaPerRoom', 'BathPerRoom', 'BuildToLand']].describe().round(2)
```

В Markdown — интерпретация каждого признака и **гипотеза**: например, «`AreaPerRoom` положительно связан с ценой — просторные дома дороже при том же числе комнат», «`Age` нелинейно: очень старые викторианские дома дорогие, середина века — дешевле». Проверим в 2.6.

2.4. Линеаризация зависимостей

Диаграммы рассеяния цели от каждого непрерывного признака:

```
cont = ['Distance', 'Landsize', 'BuildingArea', 'YearBuilt', 'Age', 'AreaPerRoom',
        'Lattitude', 'Longtitude', 'Propertycount']
fig, axes = plt.subplots(3, 3, figsize=(13, 10))
for ax, c in zip(axes.ravel(), cont):
    ax.scatter(enc[c], enc[TARGET], s=3, alpha=0.3)
    ax.set_xlabel(c); ax.set_ylabel(TARGET)
plt.tight_layout(); plt.show()
```

Ищем изгиб. Типичные находки и преобразования:

Признак	Что видно	Гипотеза	Новый признак
`BuildingArea`	рост цены замедляется, облако веером	степенной закон $y \approx Cx^{\beta}$	`log_BA` $= \ln x$ (и цель в логарифме)
`Distance`	цена быстро падает у центра, потом медленнее	$y \approx Ce^{-ax}$ или $y \approx C/x$	`inv_Dist` $= 1/(x+1)$
`Landsize`	огромный разброс по x	$\sqrt{x}$ или $\ln(1+x)$	`log_Land` $= \ln(1+x)$

```
enc['log_Price'] = np.log(enc[TARGET])
enc['log_BA'] = np.log(enc['BuildingArea'])
enc['inv_Dist'] = 1 / (enc['Distance'] + 1)
enc['log_Land'] = np.log1p(enc['Landsize'])

pairs = [('BuildingArea', 'log_BA'), ('Distance', 'inv_Dist'), ('Landsize', 'log_Land')]
fig, axes = plt.subplots(len(pairs), 2, figsize=(10, 3.2 * len(pairs)))
for (raw, new), (a1, a2) in zip(pairs, axes):
    a1.scatter(enc[raw], enc[TARGET], s=3, alpha=0.3); a1.set_xlabel(raw); a1.set_ylabel('Price')
    a2.scatter(enc[new], enc['log_Price'], s=3, alpha=0.3); a2.set_xlabel(new); a2.set_ylabel('ln
Price')
    a1.set_title(f'r = {enc[raw].corr(enc[TARGET]):.3f}')
    a2.set_title(f'r = {enc[new].corr(enc["log_Price"]):.3f}')
plt.tight_layout(); plt.show()
```

Вывод: рост $|r|$ после преобразования и «выпрямление» облака подтверждают гипотезу. Логарифм цели делает зависимость относительной: ошибка в 10 % одинаково важна для дешёвых и дорогих домов.

2.5–2.6. Корреляционный анализ

```
feats = (['Rooms', 'Distance', 'Bathroom', 'Car', 'Landsize', 'BuildingArea', 'YearBuilt',
          'Age', 'AreaPerRoom', 'BathPerRoom', 'log_BA', 'inv_Dist', 'log_Land',
          'Suburb_te', 'CouncilArea_freq'] + [TARGET, 'log_Price'])
pear = enc[feats].corr(method='pearson')
spear = enc[feats].corr(method='spearman')

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
sns.heatmap(pear, ax=axes[0], cmap='coolwarm', center=0, annot=True, fmt='.2f', annot_kws={'size':
6})
sns.heatmap(spear, ax=axes[1], cmap='coolwarm', center=0, annot=True, fmt='.2f', annot_kws={'size':
6})
axes[0].set_title('Пирсон'); axes[1].set_title('Спирмен'); plt.tight_layout(); plt.show()
```

Где коэффициенты расходятся сильнее всего:

```
diff = (spear - pear).abs()
pairs_diff = (diff.where(np.triu(np.ones(diff.shape, dtype=bool), k=1))
                  .stack().sort_values(ascending=False).head(10))
pd.DataFrame({'Пирсон': [pear.loc[a, b] for a, b in pairs_diff.index],
              'Спирмен': [spear.loc[a, b] for a, b in pairs_diff.index],
              '|разница|': pairs_diff.values}, index=pairs_diff.index).round(3)
```

Связь с целью:

```
pd.DataFrame({'Пирсон с Price': pear[TARGET], 'Спирмен с Price': spear[TARGET],
              'Пирсон с ln Price': pear['log_Price']}).drop([TARGET, 'log_Price']) \
```

```
  .sort_values('Спирмен с Price', key=abs, ascending=False).round(3)
```

Как интерпретировать:

- Спирмен заметно больше Пирсона → связь **монотонная, но нелинейная** (`Landsize`, `BuildingArea` с `Price`), или мешают выбросы.
- После линеаризации Пирсон для нового признака приближается к Спирмену: преобразование «выпрямило» зависимость. Спирмен **не меняется** от монотонных преобразований $x \mapsto \ln x$.
- Высокая корреляция признаков между собой (`BuildingArea` – `log_BA` – `AreaPerRoom`, `YearBuilt` – `Age` ≈ -1) — мультиколлинеарность, важная для ЛР5–ЛР6.

Явно ответьте: подтвердились ли гипотезы из 2.3 и 2.4.

2.7. Масштабирование признаков

Выберем два признака с разным масштабом: `Landsize` (сотни — тысячи) и `Distance` (единицы — десятки).

```
class MeanNormalizer:
    def fit(self, X):
        X = np.asarray(X, float); self.mean_ = X.mean(0); self.rng_ = X.max(0) - X.min(0); return
self
    def transform(self, X):
        return (np.asarray(X, float) - self.mean_) / self.rng_
    def fit_transform(self, X):
        return self.fit(X).transform(X)

cols = ['Landsize', 'Distance']
scalers = {'StandardScaler': StandardScaler(), 'MinMaxScaler': MinMaxScaler(),
           'RobustScaler': RobustScaler(), 'MaxAbsScaler': MaxAbsScaler(),
           'Mean normalization': MeanNormalizer()}
scaled = {name: pd.DataFrame(s.fit_transform(enc[cols]), columns=cols) for name, s in
scalers.items()}

fig, axes = plt.subplots(len(cols), len(scalers) + 1, figsize=(20, 6))
for i, c in enumerate(cols):
    sns.kdeplot(enc[c], ax=axes[i, 0]); axes[i, 0].set_title(f'{c}: исходный')
    for j, (name, d) in enumerate(scaled.items(), start=1):
        sns.kdeplot(d[c], ax=axes[i, j]); axes[i, j].set_title(f'{c}: {name}', fontsize=9)
plt.tight_layout(); plt.show()

stats_tbl = {(name, c): d[c].describe()[['mean', 'std', 'min', '50%', 'max']]
             for name, d in scaled.items() for c in cols}
pd.DataFrame(stats_tbl).T.round(3)
```

Выводы по методам:

- форма распределения **не меняется** ни одним линейным масштабированием — меняются только сдвиг и масштаб оси;

- MinMax и MaxAbs зависят от экстремумов: один большой `Landsize` сжимает все остальные значения к нулю;
- Robust (медиана/IQR) устойчив к выбросам;
- Standard даёт среднее 0 и $\sigma = 1$; удобен для градиентного спуска и регуляризации.

Выбор (пример): **StandardScaler** для всех непрерывных признаков. Обучается только на train (в ЛР5), бинарные one-hot не масштабируем.

```
to_scale = ['Rooms', 'Distance', 'Bathroom', 'Car', 'log_Land', 'log_BA', 'Age',
            'AreaPerRoom', 'BathPerRoom', 'Suburb_te', 'CouncilArea_freq']
final = enc.copy()
final[to_scale] = StandardScaler().fit_transform(final[to_scale])
final[to_scale].describe().loc[['mean', 'std']].round(3)
```

```
final.to_csv('melb_eda.csv', index=False)   # основа для ЛР5–ЛР6
```

Чек-лист

- [ ] 2.1: описание, типы, статистики, частоты.
- [ ] 2.2: способ кодирования для **каждого** категориального признака + таблица + обоснование (кардинальность, порядок, утечка).
- [ ] 2.3: ≥ 2 новых признака с интерпретацией и гипотезами.
- [ ] 2.4: scatter для **каждого** непрерывного признака; ≥ 2 преобразования, до/после.
- [ ] 2.5–2.6: две тепловые карты, таблица расхождений Пирсон/Спирмен, связь с целью, проверка гипотез.
- [ ] 2.7: ≥ 2 признака × ≥ 2 метода, KDE + статистики, выбор метода и применение.
- [ ] Restart & Run All без ошибок.